

UNIT II - REACTJS

React Components

ReactJS is a JavaScript library primarily used **for building user interfaces (UIs)** for web and mobile applications, enabling developers to create **reusable components** and dynamic, interactive experiences.

- React creates a VIRTUAL DOM in memory.

ReactJS Virtual DOM is an in-memory representation of the actual DOM (Document Object Model). React uses this lightweight JavaScript object to track changes in the application state and efficiently update the actual DOM only where necessary.

Instead of manipulating the browser's DOM directly, React creates a virtual DOM in memory, where it does all the necessary manipulating, before making the changes in the browser DOM.

- React **only changes** what needs to be changed!

React finds out what changes have been made, and changes **only** what needs to be changed.

React Versions:

Release	Released	Active Support	Security Support	Latest
19	(05 Dec 2024)	Yes	Yes	19.0.0 (05 Dec 2024)
18	(29 Mar 2022)	Ended 3 months and 1 week ago (05 Dec 2024)	Yes	18.3.1 (26 Apr 2024)
17	(20 Oct 2020)	Ended 2 years and 11 months ago (29 Mar 2022)	Yes	17.0.2 (22 Mar 2021)
16	(26 Sep 2017)	Ended 4 years ago (20 Oct 2020)	Yes	16.14.0 (14 Oct 2020)
15	(07 Apr 2016)	Ended 4 years ago (14 Oct 2020)	Yes	15.7.0 (14 Oct 2020)

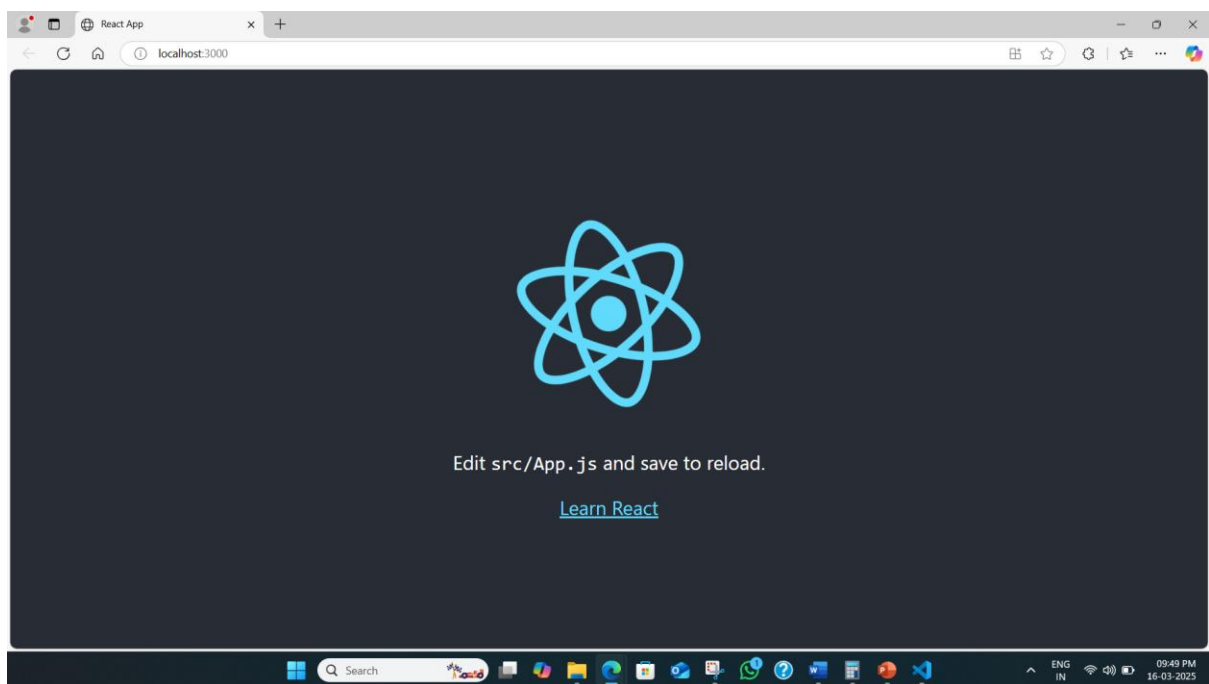
Installation and Setup:

- Install **NodeJS**.
- Create a **folder** in your **desired location for** ReactJS programs.
- In any IDE (**Visual Studio Code**), navigate to the folder created for ReactJS programs in the **terminal** using cd command.
- Type "**npx create-react-app r1**". Where "**r1**" is the user defined app name.
- The above command will create a folder named "**r1**" with all **the necessary packages**.
- **Navigate** to the "**r1**" folder through the terminal.
- Type the command "**npm start**".

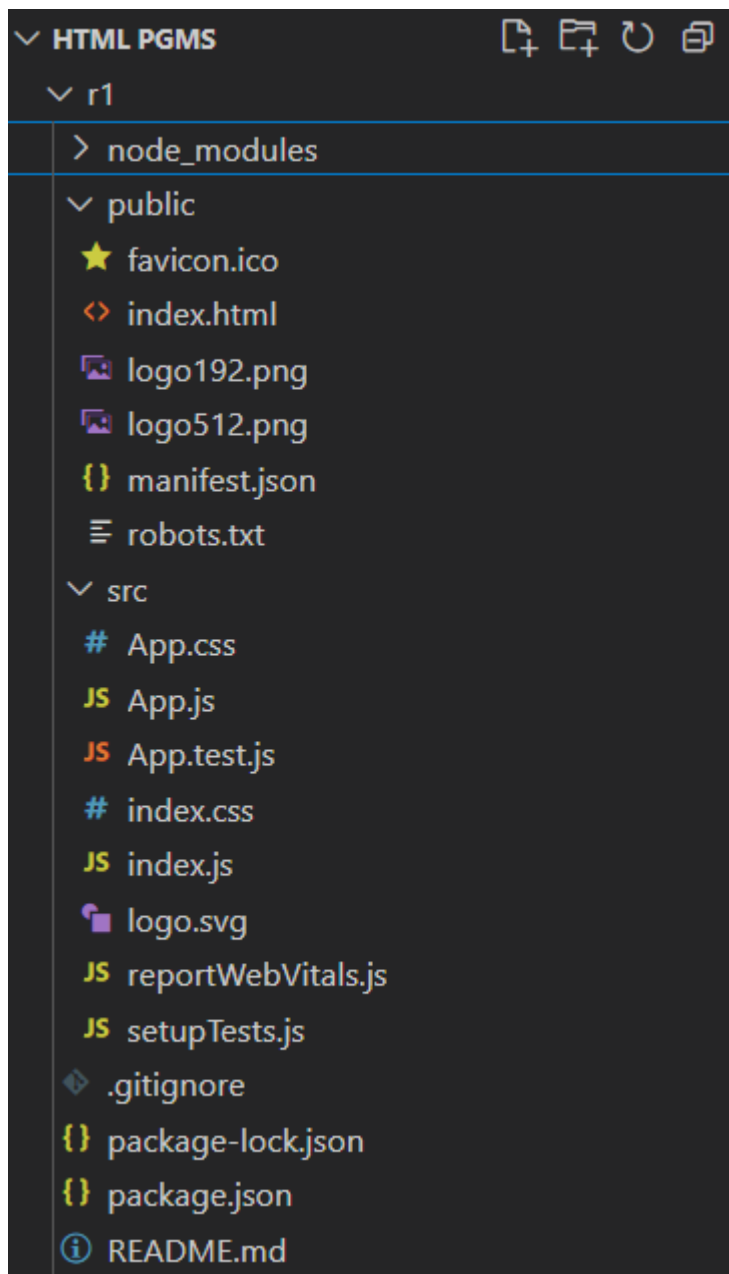
We will receive something like below

```
Compiled successfully!  
  
You can now view r1 in the browser.  
  
Local:      http://localhost:3000  
On Your Network:  http://192.168.1.33:3000  
  
Note that the development build is not optimized.  
To create a production build, use npm run build.  
  
webpack compiled successfully
```

Parallely, browser opens the above-mentioned link and displays the content shown below



The files created in the folder “r1” are shown below in IDE



- Click on “*index.js* / *App.js*”
- Remove existing code and write your code.

index.js / App.js:

```
function App()
{
  return(
    <div>
    <h1>Welcome to the First React Program</h1>
    </div>
  );
}
export default App;
```

Output:



React Components:

React components are **independent**, **reusable** building blocks in a React application that define what gets displayed on the UI. They accept **inputs called props** and **return React elements** describing the UI.

- Components can be reused across different parts of the application to maintain consistency and reduce code duplication.
- Components manage how their output is rendered in the DOM based on their state and props.
- React **loads only the necessary components**, ensuring optimized performance.
- Only the **specific Component updates** instead of the whole page.

React Components:

Functional Components:

Functional components are simpler and preferred for most use cases. They are JavaScript functions that return React elements. They are stateless, focusing on presenting UI elements without managing state or lifecycle methods. With the introduction of React Hooks, functional components can also manage state and lifecycle events. These components are ideal for scenarios where complexity is unnecessary.

Example:

```
function Car() {  
  return <h2>Hi, I am a Car!</h2>;  
}
```

Class Components:

Class components are ES6 classes that extend **React.Component**. They include additional features like state management and lifecycle methods.

When creating a React component, the component's **name must start with an upper-case letter**.

The component has to include the extends React.Component statement, this statement creates an inheritance to React.Component, and gives your component access to React.Component's functions.

The component also requires a **render()** method.

In React JSX, we commonly use div to put multiple components inside of it to return as a single component.

Example:

```
class Sample extends React.Component
{
  render()
  {
    return <h1> Welcome to Class </h1>
  }
}
```

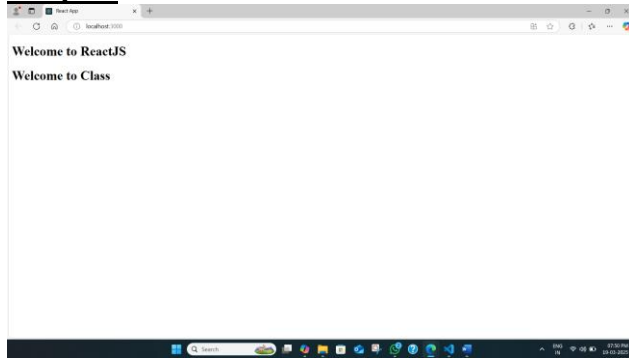
Example Program:

```
import React from 'react';
import ReactDOM from 'react-dom/client';

function Greeting() {                                // New function
  return <h1>Welcome to ReactJS</h1>;
}
class Sample extends React.Component                // New Class
{
  render(){
    return <h1>Welcome to Class</h1>;
  }
}
function M1()                                        // Function for combining multiple components
{
  return(
    <div>
      <Greeting/>
      <Sample/>
    </div>
  );
}

const root = ReactDOM.createRoot(document.getElementById('root')); // Use createRoot
root.render(<M1/>);
```

Output:



Including Components from Another File:

index.js:

```
import React from 'react';
import ReactDOM from 'react-dom/client'; // Use 'react-dom/client' for React 18
import Disp from './nf';                // Import external file (nf.js) into our file
```

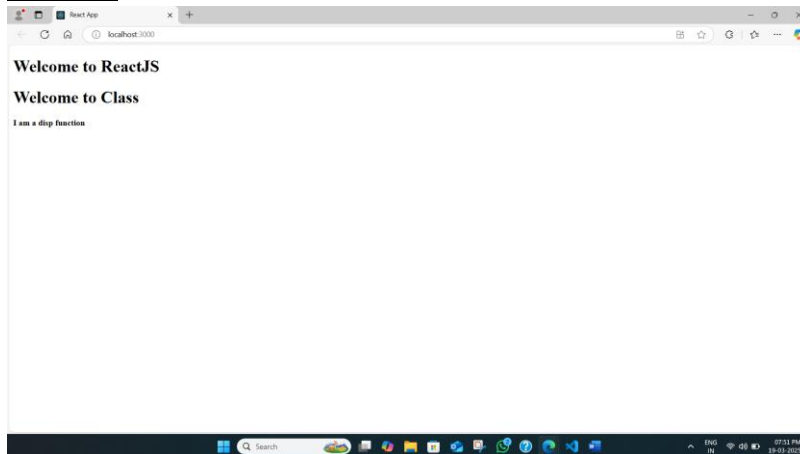
```
function Greeting() {
  return <h1>Welcome to ReactJS</h1>;
}
class Sample extends React.Component
{
  render(){
    return <h1>Welcome to Class</h1>;
  }
}
function M1()
{
  return(
    <div>
      <Greeting/>
      <Sample/>
      <Disp/>
    </div>
  );
}
```

```
const root = ReactDOM.createRoot(document.getElementById('root')); // Use createRoot
root.render(<M1/>);
```

nf.js:

```
function Disp(){
  return <h4>I am a disp function</h4>;
}
export default Disp;
```

Output:



Props:

- Props are arguments passed into React components.
- Props are passed to components via HTML attributes.

Example:

```
import React from 'react';
```

```
import ReactDOM from 'react-dom/client';
```

```
function Inf(props){  
  return <h2>Class Name = {props.a}</h2>  
}
```

```
function M1()  
{
```

```
  return(  
    <div>  
      <Inf a="II YEAR AI & DS B"></Inf>  
    </div>
```

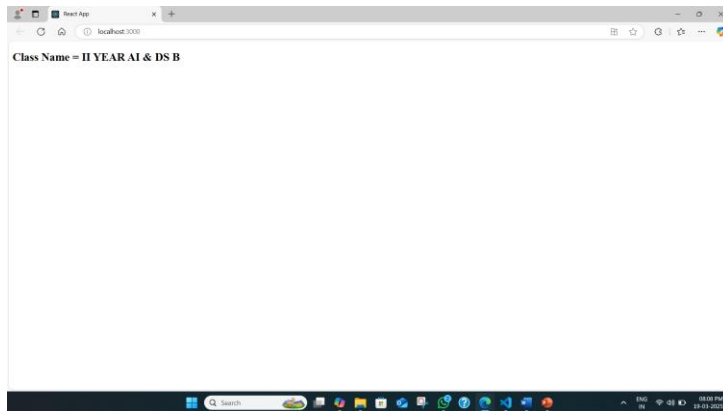
```
  );
```

```
}
```

```
const root = ReactDOM.createRoot(document.getElementById('root'));
```

```
root.render(<M1/>);
```

Output:



If you have a variable to send, and not a string as in the example above, you just put the variable name inside curly brackets:

Example:

```
import React from 'react';
```

```
import ReactDOM from 'react-dom/client'; // Use 'react-dom/client' for React 18
```

```
function Inf(props){  
  return <h2>Class Name = {props.a}</h2>  
}
```

```
function M1()  
{
```

```
  const cl="II YEAR AI & DS B";
```

```
  return(  
    <div>
```

```
    <Inf a={cl}></Inf>
```

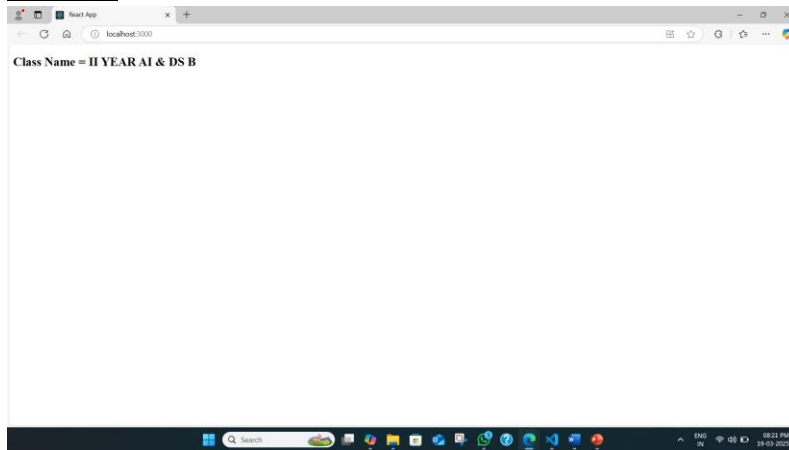
```
    </div>
```

```
  );
```

```
}
```

```
const root = ReactDOM.createRoot(document.getElementById('root')); // Use createRoot  
root.render(<M1/>);
```


Output:



Same with Objects:

Example:

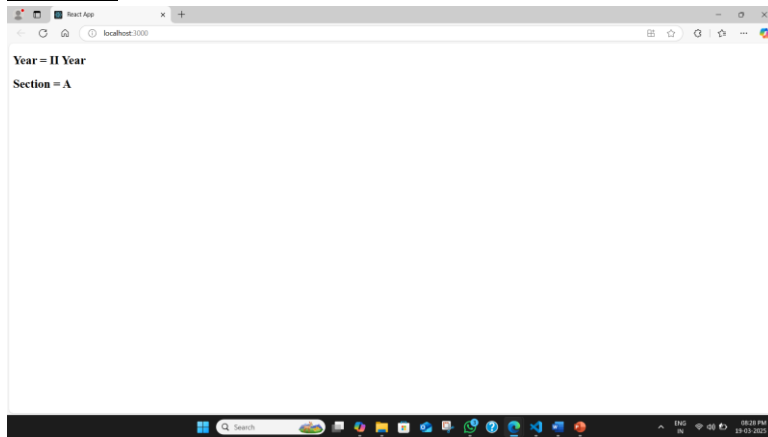
```
import React from 'react';
import ReactDOM from 'react-dom/client';
```

```
function Inf(props){
  return(
    <div>
      <h2>Year = {props.a.year}</h2>
      <h2>Section = {props.a.section}</h2>
    </div>
  );
}
```

```
function M1()
{
  const cl={year:"II Year", section:"A"};           //Objects
  return(
    <div>
      <Inf a={cl}></Inf>
    </div>
  );
}
```

```
const root = ReactDOM.createRoot(document.getElementById('root')); // Use createRoot
root.render(<M1/>);
```

Output:



Button in React:

A button may be the first interactive element that you are using in a React component.

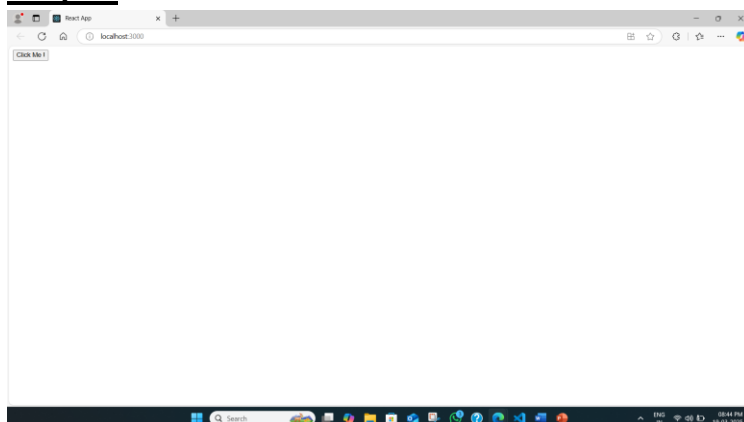
Example:

```
import React from 'react';
import ReactDOM from 'react-dom/client'; // Use 'react-dom/client' for React 18

function M1()
{
  return(
    <div>
      <button type="button">Click Me !</button>
    </div>
  );
}

const root = ReactDOM.createRoot(document.getElementById('root')); // Use createRoot
root.render(<M1/>);
```

Output:



Event Handling:

Event handling in React refers to the process of responding to user actions, such as clicks, typing, submitting a form, or hovering over elements. React provides a seamless way to handle events, ensuring that components respond dynamically to user input.

Button Click – Event Handling:

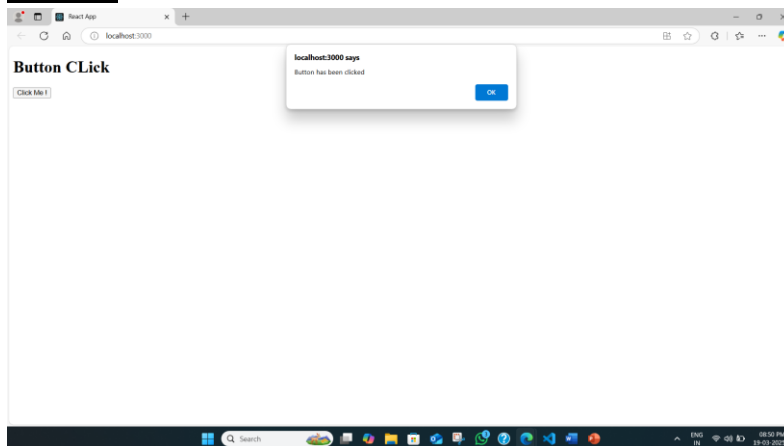
Example:

```
import React from 'react';
import ReactDOM from 'react-dom/client';

function M1()
{
  const buttonclicked=()=> {
    alert("Button has been clicked");
  }
  return(
    <div>
      <h1>Button CLick</h1>
      <button onClick={buttonclicked}>Click Me !</button>
    </div>
  );
}

const root = ReactDOM.createRoot(document.getElementById('root')); // Use createRoot
root.render(<M1/>);
```

Output:



Explanation:

Event Handler Function (buttonclicked):

- You define an arrow function, buttonclicked, inside the M1 functional component.
- This function is triggered by the onClick event of the <button>.

Rendering the Button:

- The button is set up with the onClick attribute, which links to the buttonclicked function.

Correct Use of React Features:

- You correctly use ReactDOM.createRoot() to initialize the React root in React 18.
- The M1 functional component returns valid JSX to render the title and button.

Another way – Example:

```
import React from 'react';
import ReactDOM from 'react-dom/client'; // Use 'react-dom/client' for React 18
function Buttonclicked()
{
  alert("Button has been clicked");
}
function M1()
{
  return(
    <div>
      <h1>Button Click</h1>
      <button onClick={Buttonclicked}>Click Me !</button>
    </div>
  );
}
const root = ReactDOM.createRoot(document.getElementById('root')); // Use createRoot
root.render(<M1/>);
```

Output: Same as before

React States:

- React components has a built-in state object.
- The state object is where you store property values that belong to the component.
- When the state object changes, the component re-renders.

Example:

index.js

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
```

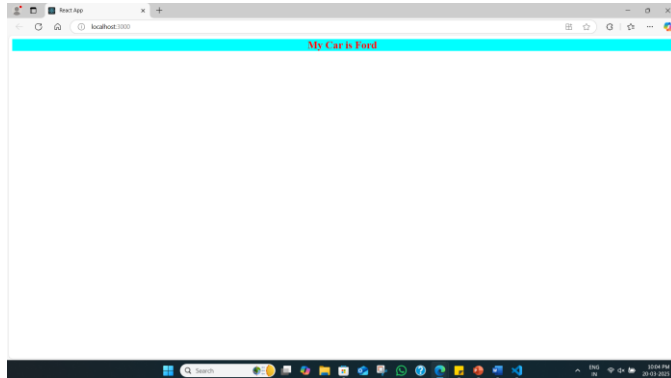
```
class Car extends React.Component {
  constructor(props) {
    super(props);
    this.state = {brand: "Ford"};
  }
  render() {
    return (
      <div>
```

```

    <h1>My Car is {this.state.brand}</h1>
  </div>
);
}
}
const root = ReactDOM.createRoot(document.getElementById('root')); // Use createRoot
root.render(<Car/>);

```

Output:



React Hooks:

Hooks were added to React in version 16.8.

Hooks allow function components to have access to state and other React features. Because of this, class components are generally no longer needed.

React useState Hook:

- The React useState Hook allows us to track state in a function component.
- State generally refers to data or properties that need to be tracking in an application.
- To use the useState Hook, we first need to import it into our component.

Example: At the top of your component, import the useState Hook.

```
import { useState } from "react";
```

Initialize useState:

We initialize our state by calling useState in our function component. useState accepts an initial state and returns two values:

- The current state.
- A function that updates the state.

There are 3 rules for hooks:

- Hooks can only be called inside React function components.
- Hooks can only be called at the top level of a component.
- Hooks cannot be conditional

Note: Hooks will not work in React class components.

Example:

index.js

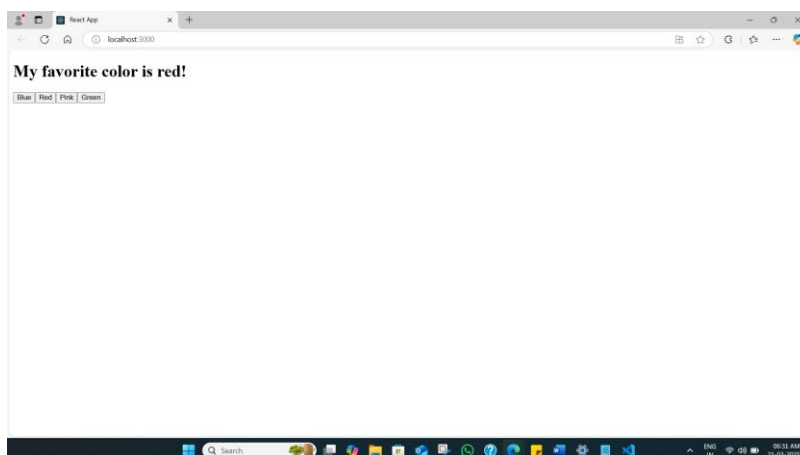
```
import React, { useState } from "react";
import ReactDOM from 'react-dom/client';

function FavoriteColor() {
  const [color, setColor] = useState("red");

  return (
    <div>
      <h1>My favorite color is {color}!</h1>
      <button type="button" onClick={() => setColor("blue")}>Blue</button>
      <button type="button" onClick={() => setColor("red")}>Red</button>
      <button type="button" onClick={() => setColor("pink")}>Pink</button>
      <button type="button" onClick={() => setColor("green")}
        >Green</button>
    </div>
  );
}

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<FavoriteColor />);
```

Output:



React Router:

React Router is a popular library used for routing in React applications. It helps manage navigation and rendering of different components based on the URL path. This enables the creation of single-page applications (SPAs) with multiple views while keeping the app fast and dynamic.

Steps to add React Router:

- To add React Router in your application, run this in the terminal from the root directory of the application:

```
npm i -D react-router-dom@latest
```

Files:

- Layout.js (For Overall Layout & Link to other pages)
- Home.js (For Home Page)
- Department.js (For Department Page)
- Contact.js (For Contact Page)
- index.js (For main file)

Example:

index.js:

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import ReactDOM from 'react-dom/client';
import Layout from './Layout';
import Home from './Home';
import Contact from './Contact';
import Departments from './Departments';
function Routerfunc()
{
  return(
    <BrowserRouter>
    <Routes>
      <Route path="/" element={<Layout />} />
      <Route index element={<Home />} />
      <Route path="departments" element={<Departments />} />
      <Route path="contact" element={<Contact />} />
    </Route>
  </Routes>
</BrowserRouter>
);
}
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Routerfunc />);
```

Explanation:

1. Imports

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import ReactDOM from 'react-dom/client';
import Layout from './Layout';
import Home from './Home';
import Contact from './Contact';
import Departments from './Departments';
```

- **React Router Imports:** import BrowserRouter, Routes, and Route from the react-router-dom library, which facilitates navigation and routing in your React app.
 - BrowserRouter is the component that enables routing based on URL changes.
 - Routes is a container for all your route definitions.
 - Route maps a specific path to a specific component.
- **ReactDOM:** The ReactDOM.createRoot API is used to render your React application into the DOM.
- **Component Imports:** Components like Layout, Home, Contact, and Departments are imported. Each of these is expected to represent a part of your application.

2. Routerfunc Component

```
function Routerfunc() {
  return(
    <BrowserRouter>
    <Routes>
      <Route path="/" element={<Layout />} />
      <Route index element={<Home />} />
      <Route path="departments" element={<Departments />} />
      <Route path="contact" element={<Contact />} />
    </Route>
  </Routes>
</BrowserRouter>
  );
}
```

Explanation:

- **BrowserRouter:** This wraps all your routes and provides the routing context for the app. It listens to URL changes and renders the appropriate components.
- **Routes Component:** This houses all the defined paths for your app.
- **Nested Routing:**

- The Route with path="/" and element={<Layout />} defines the root route (/). It loads the Layout component.
- Child routes are nested within this parent route:
 - index corresponds to the default child route under /, rendering the Home component.
 - /departments maps to the Departments component.
 - /contact maps to the Contact component.

Nested routes allow you to define paths relative to the parent path, keeping your routing organized.

3. ReactDOM.createRoot and Rendering

```
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(<Routerfunc />);
```

- ReactDOM.createRoot: Initializes the React application and attaches it to the DOM element with the ID root.
- root.render(<Routerfunc />);: Renders the Routerfunc component, which contains the routing setup, into the DOM.

4. Routing Workflow

- When a user visits /:
 - The Layout component is rendered.
 - The default index route renders the Home component inside the Layout.
- If the user navigates to /departments:
 - The Layout component is still rendered.
 - Inside it, the Departments component is displayed.
- Similarly, navigating to /contact renders the Contact component.

Layout.js:

```
import { Outlet, Link } from "react-router-dom";
const Layout = () => {
  return (
    <>
    <nav>
    <ul>
    <li>
```

```

      <Link to="/">Home</Link>
    </li>
    <li>
      <Link to="/departments">Departments</Link>
    </li>
    <li>
      <Link to="/contact">Contact</Link>
    </li>
  </ul>
</nav>

<Outlet />
</>
)
};

```

export default Layout;

Explanation:

1. Imports

```
import { Outlet, Link } from "react-router-dom";
```

Link: Used for navigation. It replaces the traditional `<a>` tags and avoids full-page reloads, ensuring your app remains a single-page application.

Outlet: Acts as a placeholder for rendering child routes. When a child route matches, its component will be displayed where the `<Outlet />` is located.

2. Navigation Menu

```

<nav>
  <ul>
    <li>
      <Link to="/">Home</Link>
    </li>
    <li>
      <Link to="/departments">Departments</Link>
    </li>
    <li>
      <Link to="/contact">Contact</Link>
    </li>
  </ul>
</nav>

```

Purpose: This navigation bar provides links to different routes (`/`, `/departments`, and `/contact`) in your application.

React Router Links: Using `<Link>` instead of `<a>` ensures efficient navigation without reloading the page. For example:

- Clicking "Home" navigates to `/` and renders the Home component.
- "Departments" navigates to `/departments`.
- "Contact" navigates to `/contact`.

3. Outlet Component

- The `<Outlet />` is crucial for **nested routes**. It dynamically renders the component of the child route that matches the current path.
- In this setup, when the Layout is rendered for the parent route (e.g., `/`), the `<Outlet />` will display:
 - The Home component for the `/` path.
 - The Departments component for `/departments`.
 - The Contact component for `/contact`.

This enables consistent rendering of the shared Layout (e.g., the navigation bar) while swapping out the child content based on the route.

Workflow:

- When the user navigates to a route:
 - The Layout component is rendered first, displaying the navigation bar.
 - Based on the route, the child route's component is injected into the `<Outlet />`.

For example:

- Navigating to `/` renders the navigation bar and the Home component inside the `<Outlet />`.
- Navigating to `/departments` renders the navigation bar and the Departments component inside the `<Outlet />`.

Summary

This Layout component is a flexible way to create a common layout for your application. You can easily expand it by adding:

- **Styling:** Use CSS for the navigation bar.
- **Additional Features:** Include a footer or a header.